UNIVERSITY INSTITUTE OF

COMPUTING

PROJECT REPORT

ON

STUDENT PERFORMANCE DATA ANALYSIS

Program Name: BCA(Hons)(Data Science)

Subject Name/Code: 25CAH-221

Submitted by:

Name: Taranjot Singh

UID: 25BCD10103

Section: 25BCD-2(A)

Submitted to:

Name: Mrs. Preeti

Designation: Assistant Professor

Sign:- _____

DATA ANALYSIS

1. TITLE PAGE

STUDENT PERFORMANCE DATA ANAL YSIS

A Mini Project / Case Study

Using Pandas, NumPy, Matplotlib and Seaborn

2. AIM

To analyze student academic performance using Python and data analysis libraries such as Pandas, NumPy,

Matplotlib and Seaborn, with a focus on marks distribution, subject-wise performance, attendance, and

student category/gender comparison.

3. OBJECTIVES

The main objectives of this project are:

1. To analyze the academic performance of students.

2. To study the distribution of marks in different subjects.

3. To compare performance across different subjects.

4. To analyze the relationship between attendance and marks.

5. To compare student performance based on gender and category.

6. To clean and preprocess the dataset using Pandas.

7. To calculate statistical measures such as mean, median and standard deviation.

8. To visualize the analyzed data using Matplotlib and Seaborn.

These objectives directly support the analysis specified in your assignment. mini project

4. PROBLEM STATEMENT

Problem Statement

Student academic performance can vary across different subjects and may be associated with factors such as

attendance and student characteristics.

The purpose of this project is to analyze student performance data and identify useful patterns in:

 Marks distribution

 Subject-wise performance

 Attendance versus marks

 Gender/category comparison

The project uses Python-based data analysis techniques to clean, analyze and visualize the dataset.

5. DATASET DESCRIPTION

Dataset Description

The dataset contains information about students and their academic performance.

The main variables used in the dataset are:

Variable Description

student_id Unique identification number of the student

gender Gender of the student

race_ethnicity Student category/group

parental_education Education level of parents

lunch Lunch category

test_preparation Test preparation status

math_score Marks obtained in Mathematics

reading_score Marks obtained in Reading

writing_score Marks obtained in Writing

attendance Student attendance percentage

The dataset contains approximately 1,005 records before data cleaning.

The numerical variables include marks and attendance, while the remaining variables are categorical.

Important: In your final Word file, you can insert a screenshot of df.head() and df.describe() after running the

code.

6. TOOLS AND TECHNOLOGIES USED

Software and Libraries

 Python

 Google Colab / Jupyter Notebook

 Pandas – Data cleaning and analysis

 NumPy – Numerical calculations

 Matplotlib – Data visualization

 Seaborn – Statistical visualization

7. COMPLETE PYTHON CODE

Use this complete code in Google Colab.

First upload:

StudentsPerformance_Final.csv

to the Colab Files section.

Then paste this code into a code cell

Code:-

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

sns.set_theme(style="whitegrid")

df = pd.read_csv("StudentsPerformance.csv")

print("=" * 60)

print("STUDENT PERFORMANCE DATA ANALYSIS")
```

```
    print("=" * 60)
    print("\nDataset Shape:", df.shape)
    print("\nDataset Columns:")
    print(df.columns.tolist())
    print("\nFirst 5 Records:")
display(df.head())
    print("\nDataset Information:")
    df.info()
    print("\nStatistical Summary:")
display(df.describe())
    print("\n" + "=" * 60)
    print("DATA CLEANING")
    print("=" * 60)
    print("\nMissing Values Before Cleaning:")
    print(df.isnull().sum())
    print("\nDuplicate Records:", df.duplicated().sum())
    # Remove duplicate records
    df = df.drop_duplicates().copy()
    # Convert numerical columns to numeric
    numeric_columns = [
   "math_score",
   "reading_score",
   "writing_score",
   "attendance"
    ]
    for column in numeric_columns:
        if column in df.columns:
            df[column] = pd.to_numeric(df[column], errors="coerce")
    # Fill missing numerical values with median
```

```
 for column in numeric_columns:
     if column in df.columns:
         df[column] = df[column].fillna(df[column].median())
 # Fill missing categorical values
 categorical_columns = [
"gender",
"race_ethnicity",
"parental_education",
"lunch",
"test_preparation"
 ]
 for column in categorical_columns:
     if column in df.columns:
         df[column] = df[column].fillna("Unknown")
 # Remove extra spaces from categorical values
 for column in categorical_columns:
     if column in df.columns:
         df[column] = df[column].astype(str).str .strip()
 print("\nMissing Values After Cleaning:")
 print(df.isnull().sum())
 print("\nDuplicate Records After Cleaning:")
 print(df.duplicated().sum())
 print("\nDataset Shape After Cleaning:", df.shape)
 df["total_score"] = (
     df["math_score"] +
     df["reading_score"] +
     df["writing_score"]
 )
 df["average_score"] = df["total_score"] / 3
```

```
    print("\nStudent Scores:")
display(
        df[
     [
        "gender",
        "math_score",
        "reading_score",
        "writing_score",
        "total_score",
        "average_score"
             ]
        ].head()
    )
    print("\n" + "=" * 60)
    print("STATISTICAL ANALYSIS")
    print("=" * 60)
    subjects = {
   "Math": df["math_score"].values,
   "Reading": df["reading_score"].values,
   "Writing": df["writing_score"].values
    }
    for subject, scores in subjects.items():
        print(f"\n{subject} Score:")
        print(
     "Mean:",
     round(np.mean(scores), 2)
        )
        print(
     "Median:",
```

```
    round(np.median(scores), 2)
        )
        print(
    "Standard Deviation:",
    round(np.std(scores), 2)
        )
        print(
    "Minimum:",
    np.min(scores)
        )
        print(
    "Maximum:",
    np.max(scores)
        )
  subject_average = {
"Math": df["math_score"].mean(),
"Reading": df["reading_score"].mean(),
"Writing": df["writing_score"].mean()
  }
  print("\nSubject-wise Average Scores:")
  for subject, score in subject_average.items():
      print(subject, ":", round(score, 2))
  plt.figure(figsize=(8, 5))
  plt.bar(
      subject_average.keys(),
      subject_average.values()
  )
  plt.title("Subject-wise Average Scores")
  plt.xlabel("Subject")
```

```
    plt.ylabel("Average Score")
    plt.ylim(0, 100)
    plt.show()
    plt.figure(figsize=(9, 5))
    sns.histplot(
        df["average_score"],
    bins=20,
    kde=True
    )
    plt.title("Distribution of Average Student Scores")
    plt.xlabel("Average Score")
    plt.ylabel("Number of Students")
    plt.show()
    gender_performance = df.groupby("gender")[
    [
     "math_score",
     "reading_score",
     "writing_score"
        ]
    ].mean()
    print("\nGender-wise Performance:")
display(
        gender_performance.round(2)
    )
    gender_performance.plot(
    kind="bar",
    figsize=(9, 6)
    )
    plt.title("Gender-wise Subject Performance")
```

```
    plt.xlabel("Gender")
    plt.ylabel("Average Score")
    plt.xticks(rotation=0)
    plt.ylim(0, 100)
    plt.show()
    category_performance = df.groupby("race_ethnicity")[
  [
     "math_score",
     "reading_score",
     "writing_score",
     "average_score"
        ]
    ].mean()
    print("\nCategory-wise Performance:")
display(
        category_performance.round(2)
    )
    if "attendance" in df.columns:
        print("\nAttendance and Average Score Analysis:")
        print(
     "Attendance Mean:",
     round(df["attendance"].mean(), 2)
        )
        print(
     "Average Score Mean:",
     round(df["average_score"].mean(), 2)
        )
```

```
        plt.figure(figsize=(8, 5))
        sns.scatterplot(
    data=df,
    x="attendance",
    y="average_score"
        )
        plt.title("Attendance vs Average Student Score")
        plt.xlabel("Attendance (%)")
        plt.ylabel("Average Score")
        plt.show()
  corr_cols = [
"math_score",
"reading_score",
"writing_score",
"average_score"
  ]
  if "attendance" in df.columns:
        corr_cols.insert(0, "attendance")
  correlation = df[corr_cols].corr()
  print("\nCorrelation Matrix:")
display(
        correlation.round(2)
  )
  plt.figure(figsize=(8, 6))
  sns.heatmap(
        correlation,
annot=True,
fmt=".2f",
```

```
    cmap="coolwarm"
    )
    plt.title("Correlation Heatmap")
    plt.show()
    plt.figure(figsize=(9, 6))
    sns.boxplot(
    data=df[
     [
        "math_score",
        "reading_score",
        "writing_score"
             ]
      ]
    )
    plt.title("Distribution of Subject Scores")
    plt.xlabel("Subjects")
    plt.ylabel("Score")
    plt.show()
    top_students = df.sort_values(
    by="average_score",
    ascending=False
    ).head(10)
    print("\nTop 10 Students:")
display(
        top_students[
     [
        "gender",
        "math_score",
        "reading_score",
```

```
        "writing_score",
        "average_score"
            ]
        ].round(2)
    )
    if "attendance" in df.columns:
        high_attendance = df[
            df["attendance"] >= 90
        ]
    print("\nStudents with Attendance >= 90:")
    display(
            high_attendance.head()
        )
    high_performance = df[
        df["average_score"] >= 80
    ]
    print("\nStudents with Average Score >= 80:")
display(
        high_performance.head()
    )
    print("\n" + "=" * 60)
    print("FINAL PROJECT SUMMARY")
    print("=" * 60)
    print(
    "Total Students:",
    len(df)
    )
    print(
    "Average Math Score:",
```

```
    round(df["math_score"].mean(), 2)
    )
    print(
    "Average Reading Score:",
    round(df["reading_score"].mean(), 2)
    )
    print(
    "Average Writing Score:",
    round(df["writing_score"].mean(), 2)
)
    print(
    "Overall Average Score:",
    round(df["average_score"].mean(), 2)
    )
    if "attendance" in df.columns:
        print(
     "Average Attendance:",
     round(df["attendance"].mean(), 2)
        )
    print(
    "Highest Average Score:",
    round(df["average_score"].max(), 2)
    )
    print(
    "Lowest Average Score:",
    round(df["average_score"].min(), 2)
    )
    print("\nAnalysis Completed Successfully!")
```

7. Data Cleaning

 Checked for missing values.

 Removed duplicate records.

 Converted numerical columns into appropriate data types.

 Filled missing numerical values using the median.

 Filled missing categorical values with "Unknown".

 Removed unnecessary spaces from categorical values.

8. Data Analysis

 Mean, median and standard deviation of subject scores.

 Subject-wise average performance.

 Gender-wise and category-wise comparison.

 Attendance vs average marks.

 Correlation between attendance and scores.

 Sorting and filtering of student records.

9. Screenshort:-

9.Reference:- https://github.com/sc582752-eng/python_project/blob/main/UID - 25BCD10093.docx

10. Data Visualization

The following visualizations were used:

 Bar chart

 Histogram

 Scatter plot

 Heatmap

 Box plot

11. Conclusion

The project successfully analyzed student academic performance using Python and its data analysis libraries.

The dataset was cleaned and analyzed using Pandas and NumPy, while Matplotlib and Seaborn were used for

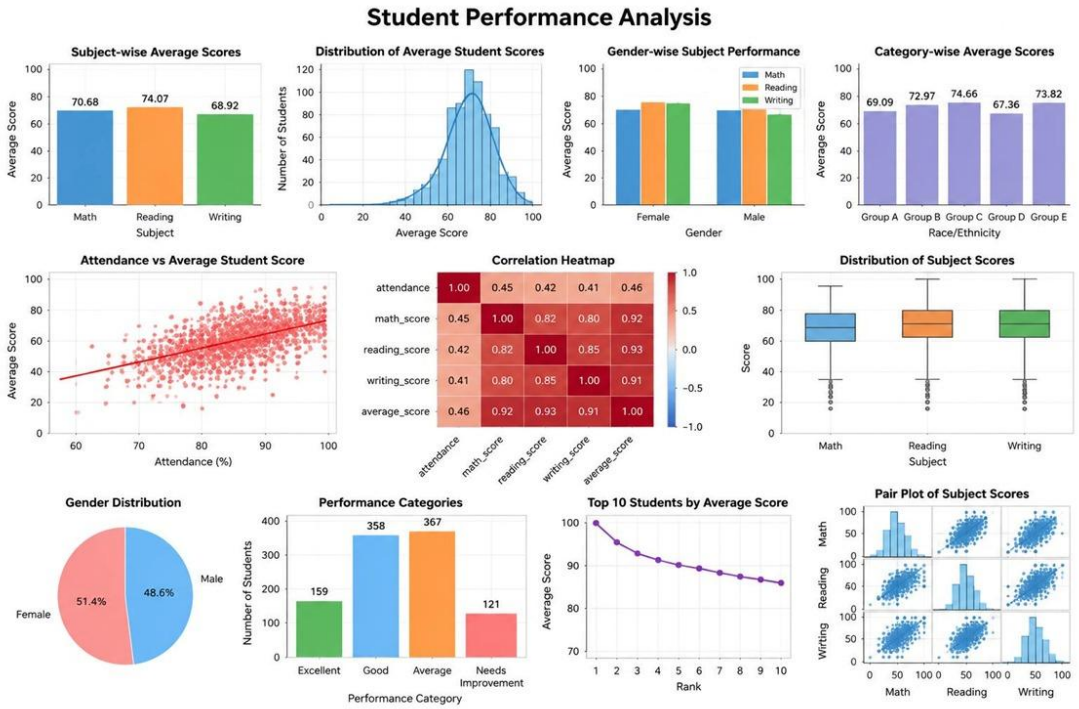
visualization. The analysis helped compare subject-wise performance, examine attendance and marks, and

identify relationships between different factors. Overall, the project demonstrates how data analysis and

visualization can be used to understand patterns in student academic performance.

https://github.com/TaranDeol578/python-mini-project

Original Charts and Visuals



GIT HUB :- https://github.com/TaranDeol578/python-mini-project